Architecture & Trade-offs
In-Memory Account Ledger Core

Author: Sandeep Badikolu
Repository: https://github.com/sandeep-badikolu/InMemoryAccountLedger
Mal Bank - Staff Software Engineer exercise (Part 2)

1. Append-only at scale

What breaks at 100x volume? The engine scans the full in-memory event log for each reporting day (O(days x events x accounts)). At 100x volume, CPU and heap grow linearly with stream length; interactive replay and batch close become too slow.

Unbounded state: (1) the immutable event log plus system-generated fee/interest events, (2) per-replay snapshot materialization, (3) authorization indexes in production.

Cheapest structural fix: segmented append-only log with immutable boundary snapshots (per-day or per-month closing balance and hold summary). Replay loads one summary plus only the tail segment - still append-only, no mutation of historical events.

Secondary optimizations (after segmentation): maintain in-memory indexes by account and value_date, stream processing instead of full rescans, and async projection workers that update read models used by statements and channels.

2. Value-dated entries (UAE-licensed bank)

Backdated debits change historical closes and can restate fees, interest, customer statements, and regulatory returns. UAE context: CBUAE supervisory accuracy, AML audit trail, and customer fairness on balances/charges.

Control before go-live: maker-checker with effective-date policy - block or queue value-dated posts before current business date unless second-line approval; store approver, reason, and customer impact; run deterministic restatement jobs for affected fees/interest/reports using compensating entries only (original events remain immutable).

Operational surfaces include: customer mobile/web balances, end-of-day GL bridge, liquidity reporting, and dispute investigation. Each must declare which value_date convention it uses (booked date vs effective date) to avoid silent mismatch.

3. Authorization lifecycle beyond settlement

Expiry (hotels/fuel pre-auth TTL) - release hold, no debit. Partial capture - debit actual, release remainder. Void before capture - idempotent void. Incremental auth - linked holds with total cap. Chargeback - separate dispute workflow. Stale holds (Auth-B never settled) - TTL, reporting, and escalation. Production must not assume settlement is the only terminal state.

4. Scope trade-offs

In-memory only - risk: no DR/durability. Single-process replay - risk: concurrent writers. Fee recompute after full stream - risk: fees differ from point-in-time posting. Recorded-day auth rules - risk: vs real-time available balance. No idempotency keys - risk: duplicate delivery. Day-6 interest only - risk: no mid-period accrual payouts. No FX/API - risk: integration and fraud controls deferred.

5. Link to Part 1 implementation

The GitHub repo provides immutable events, E6 rejection, holds vs ledger balance, NUMBERS/AMBIGUITIES/REJECTED/WORKLOG, and one intentional failing test documenting the fee vs reversal

policy conflict. Production would externalize the log, serialize per-account commands, and treat fees/interest as first-class posted entries with explicit reversal policy.